

Project Report

NETWORK RULE ENGINE USING OOP IN C++
MUHAMMAD AFAN TARIQ

Contents

Introduction	3
Motivation and Objectives	3
Importance of Object-Oriented Programming (OOP)	3
Core Functionality and Features	4
Scope	5
Limitations	5
Object-Oriented Programming Concepts Used	6
Classes and Objects	6
Inheritance	7
Polymorphism	7
Encapsulation	7
Abstraction	7
Error Handling	7
Preventing Crashes	7
Input Validation Mechanisms	8
Exception Handling Techniques	8
Security Features	9
Secure Coding Practices	9
Data Validation and Memory Safety	9
Authentication and Access Control	10
Code Organization & Documentation	10
Structure of the Source Code	10
Graphical User Interface (GUI)	12
Implemented GUI Components	12
TabControl:	12
User Interactions and Workflow	13
Novelty & Innovation	14
Additional Features Enhancing Usability	14
Conclusion	15
Project Achievements	15
Challenges	16
Potential Improvements and Future Enhancements	16

Visual Representation	17
References	19
External Libraries.....	19
NET Framework / Windows Forms	19
C++/CLI Interop	19

Introduction

This project is a user-friendly Network Rule Engine application developed using C++/CLI and Windows Forms. The system provides an intuitive graphical user interface (GUI) that allows users to define, manage, and apply custom rules for monitoring and controlling network traffic.

The goal of the application is to offer a secure, interactive, and efficient environment for users whether administrators or developers—to monitor network behaviour, enforce traffic policies, and enhance system security through customizable rules and real-time packet analysis.

Motivation and Objectives

Motivation

Modern networks demand flexible and user-friendly tools for managing traffic, enforcing security policies, and analysing network behaviour. Traditional manual configurations are often error-prone, lack transparency, and offer limited control. While some existing solutions address parts of the problem, they often fall short in terms of customization, usability, or integration with other tools. This project aims to bridge that gap by offering a more tailored and accessible solution.

Objectives

- Develop an intuitive interface for creating, editing, and organizing network rules.
- Enable real-time packet capture and detailed logging to monitor network activity effectively.
- Implement rule-based analysis to evaluate captured packets against custom traffic rules.
- Ensure secure storage and encryption of sensitive log data and rule files.
- Support easy import and export of rules for portability, sharing, and backup.

Importance of Object-Oriented Programming (OOP)

Object-Oriented Programming plays a crucial role in the design and development of this application. It provides a clear and structured way to organize code, making the system more robust, scalable, and easier to manage.

Modularity

The system is divided into well-defined components such as Rule, TimeBasedRule, Packet, RuleEngine, and NetworkPacketCapture. Each class encapsulates its own data and logic, promoting a clean separation of responsibilities and simplifying both development and debugging.

Reusability

Core functionalities like rule matching and packet handling are implemented once and reused throughout the application. This minimizes code duplication and ensures consistency across the system.

Extensibility

The design supports easy expansion. For example, new rule types or analysis capabilities can be added by extending existing base classes, such as inheriting from the Rule class, without altering the core structure.

Maintainability

Through encapsulation and clearly defined interfaces, the system remains easier to understand, test, and update. This helps reduce bugs and improves long-term maintainability as the project evolves.

Core Functionality and Features

The Network Rule Engine is a Windows-based tool that empowers users to monitor, manage, and analyze network traffic using a customizable set of rules. It combines real-time packet capture with rule-based evaluation and secure logging, all accessible through a user-friendly graphical interface.

Rules Management

- Create, view, edit, remove, save, and load network rules.
- Supports both default (built-in) rules and user-defined rules imported from files.
- Rules can match source/destination IP addresses, ports, protocols, actions (ALLOW, DENY, LOG), priorities, and optional time constraints using TimeBasedRule.

Packet Capture

- Lists all available network interfaces and allows selection for live capture.
- Captures a specified number of packets from the chosen device.
- Each packet is automatically evaluated against the current rules, and the resulting actions are logged.
- Optional encryption of logs using a user-supplied key for added security.

Packet Analysis

- Load and analyse previously captured packet data from CSV, LOG, or encrypted DAT files.
- Evaluates packets using the current rule set and displays analysis results within the GUI.

System Logging

- Tracks and logs significant system events such as rule modifications, capture events, and errors.
- Provides options to view, clear, and save logs for audit or troubleshooting purposes.

User Interface

- Tabbed layout for seamless navigation between different modules (Rules, Capture, Analysis, Logs).
- Uses data grids for rule visualization and editing.
- Dialog windows facilitate file operations and secure key input for encryption.

Scope

Intended Use

- Designed for educational, research, or small-scale administrative use.
- Ideal for users who need to define, test, and analyse network rules in a controlled environment.

Supported Rule Types

- **Basic rules:** Match on IP addresses, ports, protocols, actions, and priorities.
- **Time-based rules:** Active only during specified hours.

File Formats

- Supports CSV, LOG, and encrypted DAT formats for rules and log files.

Encryption

- Implements basic file encryption using a user-defined key for sensitive rule and log data.

Limitations

Network Device Support

- Depends on the system's available network interfaces.
- Packet capture may require administrative privileges and may behave differently based on system configuration.

Rule Complexity

- Rules are limited to predefined fields and logic as implemented in the Rule and TimeBasedRule classes.
- Advanced features like deep packet inspection or application-layer filtering are not supported.

Performance

- Optimized for moderate traffic volumes and small-to-medium rule sets.
- Not designed for enterprise-level or high-throughput network environments.

Security

- Encryption is basic and intended for demonstration or lightweight security needs.
- Not suitable for protecting highly sensitive data in production environments.

Extensibility

- While the object-oriented structure allows for future expansion, only the built-in rule types and actions are supported in the current version.

Object-Oriented Programming Concepts Used

Classes and Objects

Rule

Represents a single network rule, including fields for source/destination IP, ports, protocol, action, and priority. Provides methods for matching packets and exporting rule data.

TimeBasedRule

Extends the Rule class by adding time-based conditions. These rules are only active during specified hours, making them useful for scheduling rule enforcement.

Packet

A simple class that models a network packet with details such as source/destination IP, ports, protocol, and size. It's used during both live capture and file-based analysis.

RuleEngine

Acts as the brain of the rule evaluation system. It stores all the rules, checks incoming packets against them, and manages rule-related operations.

NetworkPacketCapture

Handles the technical side of packet capturing, such as listing available network devices and capturing traffic from the selected one.

MainForm

The main window of the application, which connects all components and manages the user interface. It controls how users interact with rules, captures packets, views logs, and more.

Inheritance

- The TimeBasedRule class inherits from Rule, so it gets all the basic rule functionality and adds its own time-based logic on top.
- Other parts of the system like RuleEngine and NetworkPacketCapture are used inside other classes (composition), rather than using inheritance, which keeps things modular and flexible.

Polymorphism

Polymorphism allows the system to treat different types of rules in a consistent way while still running the correct logic behind the scenes:

- The Rule class provides general methods like matches(), display(), and toCsvString(), which are marked as virtual.
- The TimeBasedRule class overrides these methods to include its own specific logic for handling time-based conditions.

Encapsulation

Encapsulation ensures that each class controls how its internal data is accessed and changed:

- Most class variables (like rule IDs or IP addresses) are kept private.
- Public getter and setter methods are provided to read or update those values safely.

Abstraction

- The Rule class gives a simple interface for checking if a packet matches a rule, while hiding how that logic works internally.
- The Packet class shows only the essential packet data needed for analysis, not the full raw structure.
- The MainForm class wraps all the behind-the-scenes complexity—like rule evaluation and packet capture—into an easy-to-use interface for the user.

Error Handling

Preventing Crashes

Try-Catch Blocks

Key operations that might throw exceptions such as file reading/writing, encryption processes, or network device selection are wrapped in try-catch blocks. This allows the program to catch and handle errors gracefully, rather than terminating abruptly.

Null and State Checks

Before interacting with UI elements or internal data, the application checks for valid states. For example, it verifies that a rule row is selected before attempting to delete it, or that a

network device is chosen before starting packet capture. This helps avoid invalid operations and runtime errors.

Safe Accessors

Functions like `GetCellValueSafe` are used to safely retrieve data from GUI components such as data grids. These methods ensure null values are handled correctly by returning default values (e.g., an empty string), avoiding potential null reference exceptions.

Input Validation Mechanisms

Duplicate Rule ID Checks

Before saving rule sets, the system checks for duplicate rule IDs. If duplicates are found, the user is alerted, and the save operation is halted until the issue is resolved.

Required Field Checks

The application ensures that essential fields (e.g., selected device, file path) are filled before executing actions like packet capture or file analysis. If required fields are missing, the user is prompted to provide the necessary input.

Encryption Key Validation

When dealing with encrypted rule or log files, the system verifies whether an encryption key has been provided. If not, it prompts the user to enter one and cancels the operation to prevent data loss or corruption.

Exception Handling Techniques

User Feedback

Instead of crashing, the application notifies the user with clear error messages using `MessageBox::Show`. These messages explain what went wrong and often suggest how to fix the issue.

Graceful Abortion of Operations

If a critical error is detected such as a missing file or encryption key the current task is safely stopped without affecting other parts of the application. This ensures the rest of the system continues to function normally.

Logging

All significant errors and events are recorded using the `LogMessage` method. These logs include timestamps and descriptive messages, allowing developers or administrators to review and troubleshoot problems effectively.

Security Features

Secure Coding Practices

Exception Handling

All sensitive operations such as file I/O, encryption/decryption, and network device enumeration are enclosed in try-catch blocks. This not only prevents unexpected crashes but also ensures that sensitive data isn't exposed through unhandled errors.

Sensitive Data Handling

When working with encrypted rule or log files, users are prompted to enter an encryption key. This key is never stored in plain text, and encryption is handled through the `BasicFileEncryptor` class. This ensures that even if files are accessed outside the application, their contents remain protected.

Resource Management

Objects like `RuleEngine` and `NetworkPacketCapture` are dynamically created and properly cleaned up in destructors. This avoids memory leaks and ensures that resources are released when no longer needed.

Data Validation and Memory Safety

Input Validation

Rule IDs: Before saving, the application checks for duplicate rule IDs and blocks the action until the issue is resolved.

Required Fields: Before performing key operations like starting a capture or saving encrypted files, the system checks that essential inputs (e.g., file paths, selected device, encryption key) are provided.

Safe Grid Access: The `GetCellValueSafe` function ensures that UI elements like data grids are accessed without risking null reference errors.

Boundary Checks

- File parsers skip empty lines and headers to avoid corrupt or malformed data.
- Packet capture functions retry when data is incomplete or fails to load, helping to avoid errors in analysis and logging.

Memory Safety

- Smart pointers (like `std::unique_ptr`) are used in key areas such as rule storage, helping to automatically manage memory and prevent leaks or dangling pointers.

- The system also includes checks like verifying `isCapturing` before stopping threads, to ensure operations complete safely and predictably.

Authentication and Access Control

User Prompts for Sensitive Actions

Whenever users attempt to save or load encrypted files, they are required to enter an encryption key via a secure prompt. If no key is provided, the operation is canceled, ensuring that encrypted content isn't exposed or mishandled. The application never stores or hardcodes the key, reducing the risk of accidental leakage.

No Built-in Authentication

This version does not include user accounts or role-based access control. All users have equal access to features and data within the system, which is acceptable for the intended use cases (educational and small-scale environments).

System Permissions

Certain operations, like accessing network interfaces, may require administrative privileges. The application detects insufficient permissions and informs the user, avoiding silent failures or security bypasses.

Code Organization & Documentation

Structure of the Source Code

Modular Design

The project is organized into well-defined header (.h) and source (.cpp) files, with each file focused on a specific functionality or system component. This modularity helps in isolating concerns and improving reusability:

Core Components:

- **MainForm.h/.cpp:** Manages the graphical user interface and event-driven logic.
- **Rule.h/.cpp, TimeBasedRule.h/.cpp:** Define the base rule structures and time-based extensions.
- **RuleEngine.h/.cpp:** Contains the logic for managing, evaluating, and applying rules.
- **Packet.h/.cpp:** Models network packets for analysis and matching.
- **NetworkPacketCapture.h/.cpp:** Handles listing of devices and live packet capture.
- **BasicFileEncryptor.h/.cpp:** Implements file encryption and decryption functionality.
- Additional supporting files provide logging, conflict resolution, and utility interfaces.

Class-Based Architecture:

Each major feature is implemented as a standalone class. The GUI (MainForm) acts as a controller, interacting with backend components like RuleEngine and NetworkPacketCapture to perform user-initiated tasks.

Separation of Concerns:

Responsibilities are cleanly split between user interface logic, business logic (rule handling), and utility services (logging, encryption), which enhances readability and makes future updates easier to manage.

Naming Conventions and Commenting Practices

Naming Conventions:

- **Classes:** Use PascalCase (e.g., MainForm, RuleEngine, TimeBasedRule).
- **Methods/Functions:** Use camelCase or PascalCase depending on platform conventions (e.g., InitializeDefaultRules, btnAddRule_Click, getNextAvailableId).
- **Variables:** Use camelCase for both local and member variables (e.g., isCapturing, engine, packetCapture).
- **UI Controls:** Prefix with control type (e.g., btnAddRule, dgvFileRules, rtbSystemLog) for clarity and easier navigation.

Commenting Practices:

- **File Headers and Regions:** Use #pragma once and #region directives to manage designer-generated code.
- **Class/Method Documentation:** XML-style comments (/// <summary>) describe the purpose of classes and methods, aiding IntelliSense and documentation generation.
- **Inline Comments:** Highlight important logic, workaround decisions, or potential caveats—such as retry logic for unstable packet capture.
- **TODO Notes:** Mark areas for future improvement or explain specific design decisions (e.g., use of helper functions to avoid redundancy).

Readability and Maintainability

- **Consistent Formatting:** Uniform indentation, spacing, and brace placement ensure a clean and readable layout.
- **Logical Grouping:** Related methods and variables are grouped together—for instance, event handlers, helper functions, and UI control declarations—making the code easier to scan and modify.
- **Descriptive Naming:** Functions and variables are clearly named to reflect their purpose, reducing reliance on excessive comments.
- **Utility Functions:** Repetitive tasks, such as safely accessing DataGridView cells, are encapsulated in helper functions like GetCellValueSafe, promoting code reuse and reducing errors.

- **Robust Error Handling:** All exceptions and unexpected scenarios are managed gracefully, with informative messages shown to the user and detailed logs saved to assist in debugging.

Graphical User Interface (GUI)

Implemented GUI Components

The application uses a tabbed Windows Forms interface, organized as follows:

TabControl:

Main container with four tabs:

Rules Management (tabRules)

This tab enables users to view, add, remove, and manage network rules.

DataGridViews:

- **dgvDefaultRules:** Displays built-in system rules (read-only).
- **dgvFileRules:** Editable grid for user-defined rules loaded from or saved to files.

Buttons:

- **btnAddRule:** Adds a new rule to dgvFileRules.
- **btnRemoveRule:** Deletes the selected rule from the file rules grid.
- **btnSaveRules:** Saves rules to a file (CSV, LOG, or encrypted DAT).
- **btnLoadRules:** Loads rules from a user-selected file into the grid.

Labels:

- **lblDefaultRules:** Title for the default rules section.
- **lblFileRules:** Title for the user-defined rules section.

Packet Capture (tabCapture)

This section allows users to capture live network traffic from available interfaces.

ComboBox:

- **cmbNetworkDevices:** Lists available network devices for packet capture.

Buttons:

- **btnRefreshDevices:** Refreshes the list of available devices.
- **btnStartCapture:** Begins capturing packets based on user settings.
- **btnStopCapture:** Stops the active capture process.

Other Controls:

- **numPacketsToCapture:** Sets the number of packets to capture.

- **chkEncryptLogs**: Toggles encryption for captured logs.
- **txtEncryptionKey**: Accepts the user-supplied encryption key (enabled only if encryption is selected).
- **rtbCaptureLog**: Displays real-time capture logs.

Packet Analysis (tabAnalysis)

Enables post-capture analysis of packet data files.

Buttons:

- **btnBrowsePacketFile**: Opens a file dialog for selecting a packet file.
- **btnAnalyzeFile**: Analyzes the selected file using the current rules.

Other Controls:

- **txtPacketFilePath**: Displays the selected file path.
- **rtbAnalysisResults**: Shows the results of the analysis in a read-only text box.

System Logs (tabLogs)

Provides visibility into internal operations and error messages.

TextBox:

- **rtbSystemLog**: Displays system-generated logs and events.

Buttons:

- **btnClearLogs**: Clears the log output.
- **btnSaveLogs**: Saves logs to a user-specified file for archiving or troubleshooting.

User Interactions and Workflow

Rules Management:

- View default rules in a protected grid.
- Add, edit, and delete custom rules via an editable grid.
- Save/load rule sets from supported formats (CSV, LOG, encrypted DAT).
- Automatic validation prevents duplicate rule IDs before saving.

Packet Capture:

- Select a network device and specify the number of packets to capture.
- Start/stop live capture with real-time logging.
- Optionally encrypt logs using a secure user-supplied key.

Packet Analysis:

- Load a previously captured packet file.
- Analyze it using active rules and display results clearly.

System Logs:

- Record all significant user actions, errors, and system messages.
- Enable users to clear logs or save them for auditing or diagnostics.

Novelty & Innovation

Time-Based Rule Matching:

- **The TimeBasedRule** class extends the standard Rule class by incorporating time constraints, enabling rules that are only active during specific hours.
- **Example** Use Case: Allowing FTP traffic only during business hours for security and compliance.
- **Innovation:** This level of temporal control is rarely found in lightweight or entry-level rule engines.

Flexible Rule Storage and Encryption:

- **Supports saving and loading rules in multiple formats:** .CSV, .LOG, and encrypted DAT.
- **Uses the BasicFileEncryptor** to encrypt/decrypt files, ensuring sensitive configurations remain secure even if stored on shared or unprotected systems.
- **Benefit:** Balances usability with data protection without requiring complex third-party libraries.

Real-Time, Threaded Packet Capture:

- **Packet capture** is handled in a background thread to avoid freezing the GUI.
- **User Impact:** Users can stop capture, monitor logs, or manage rules simultaneously without experiencing UI lag.

Safe and Robust Data Handling:

- Helper functions like GetCellValueSafe and extensive input validation (e.g., duplicate ID checks, null validation) ensure application stability.
- **Reliability:** Minimizes runtime errors and protects against user input mistakes.

Additional Features Enhancing Usability

Tabbed, Intuitive GUI:

Interface divided into four functional tabs:

- Rules Management
- Packet Capture
- Packet Analysis
- System Logs

Result: Simplifies navigation and separates concerns for users.

Editable and Read-Only Rule Grids:

- System rules are protected in a read-only grid.
- User-defined rules are editable, allowing customization while preventing accidental changes to critical rules.

Immediate User Feedback:

- All major actions (e.g., rule edits, capture status, file operations) are logged in real time and displayed in the System Logs tab.
- **Advantage:** Keeps users informed and aids in troubleshooting.

Encryption Key Prompting:

- When handling encrypted files, the system prompts users for a key if one is not already supplied.
- **Security Benefit:** Prevents accidental data exposure due to missing encryption input.

Device Discovery and Selection:

- Automatically enumerates available network devices.
- Users can select the appropriate interface for capture, accommodating systems with multiple adapters.

Graceful Error Handling:

- Exception handling is implemented around all major operations (file I/O, encryption, network access).
- **Outcome:** User-friendly error messages and detailed log entries reduce frustration and improve fault recovery.

Conclusion

Project Achievements

- **Comprehensive Rule Management:**
The system allows users to manage both default and custom network rules, including support for advanced time-based rules, enabling context-aware policy enforcement.
- **Secure and Flexible Data Handling:**
Rules and logs can be saved and loaded in CSV, LOG, or encrypted DAT formats. The built-in encryption functionality helps protect sensitive configurations and traffic data.
- **Real-Time Packet Capture and Analysis:**
Live packet capture on user-selected devices is supported, with real-time evaluation

against rules. The use of background threads ensures the GUI remains responsive throughout the capture process.

- **User-Friendly Interface:**
A tabbed Windows Forms interface provides logical organization of features, simplifying navigation across rule management, packet capture, analysis, and logging.
- **Robust Error Handling:**
Extensive input validation, safe access patterns, and detailed error logging reduce the risk of crashes and guide users through resolving common issues.

Challenges

- **Device Compatibility and Permissions:**
Packet capture depends on system-level permissions and compatible network devices, which can vary across user environments and potentially limit functionality.
- **Rule Complexity Limitations:**
While basic and time-based rules are supported, more advanced filtering techniques—such as deep packet inspection or Layer 7 filtering—are not yet implemented.
- **Encryption Simplicity:**
The current file encryption mechanism is functional but may not meet stricter compliance or enterprise-grade security requirements.
- **Scalability:**
Designed for moderate usage, performance may degrade with very large rule sets or high-throughput network environments.

Potential Improvements and Future Enhancements

- **Advanced Rule Types:**
Extend the rule engine to support richer criteria, including protocol-specific attributes, application-layer data, and custom scripting.
- **Enhanced Security:**
Upgrade encryption to use industry-standard algorithms and consider adding user authentication or access control for sensitive actions.
- **Performance Optimization:**
Introduce parallel rule evaluation, indexing, or caching mechanisms to improve scalability and responsiveness.
- **Automated Rule Conflict Detection:**
Implement intelligent conflict detection and visualization tools to help users resolve overlapping or contradictory rules effectively.
- **Reporting and Visualization:**
Add dashboards or charts for analyzing traffic patterns, rule matches, and security trends to improve usability and monitoring.
- **Cross-Platform Support:**
Explore options for Linux support or develop a web-based version of the application to increase portability and accessibility.

Visual Representation

Network Rule Engine

Rules Management | Packet Capture | Packet Analysis | System Logs

System Default Rules

ID	Name	Source	Destination	Src Port	Dest Port	Protocol	Action	Priority	Time
1	Allow Corp Web ...	ANY	10.0.1.0/24	-1	443	TCP	ALLOW	50	
2	Allow Corp Web ...	ANY	10.0.1.0/24	-1	80	TCP	ALLOW	40	
3	Allow FTP (Work ...	ANY	ANY	-1	62774	TCP	ALLOW	30	9.23
4	Block Bad Site	ANY	6.6.6.6	-1	-1	ANY	DENY	100	
5	Override Block B...	ANY	6.6.6.6	-1	-1	ANY	ALLOW	110	
6	Log All Web to C...	ANY	10.0.1.0/24	-1	443	TCP	LOG	45	
7	Allow Local DNS	ANY	ANY	-1	53	UDP	ALLOW	60	
8	Allow Local DNS	ANY	ANY	-1	53	TCP	ALLOW	60	
9	Allow DHCP	ANY	ANY	68	67	UDP	ALLOW	70	

Rules From File

ID	Name	Source	Destination	Src Port	Dest Port	Protocol	Action	Priority	Time
1	Allow HTTPS	172.16.0.0/16	10.0.1.0/24	-1	-1	UDP	LOG	37	0.23
2	Block Telnet	172.16.0.0/16	192.168.100.0/24	-1	25	TCP	LOG	76	0.23
3	Log FTP	10.0.0.0/8	6.6.6.6	-1	53	TCP	LOG	18	18.6
4	Allow DNS	172.16.0.0/16	8.8.8.8	-1	443	TCP	DENY	95	
5	Block SSH	192.168.1.0/24	10.0.1.0/24	-1	-1	UDP	DENY	43	0.23
6	Allow Mail	192.168.1.0/24	192.168.100.0/24	-1	443	TCP	LOG	38	9.17
7	Deny ICMP	172.16.0.0/16	8.8.8.8	-1	445	UDP	DENY	64	
8	Log Admin Access	172.16.0.0/16	ANY	-1	443	ANY	ALLOW	79	9.17
9	Allow Web	192.168.1.0/24	ANY	-1	3389	TCP	ALLOW	28	0.23
10	Block Malware Site	10.0.0.0/8	192.168.100.0/24	-1	0	UDP	LOG	44	
11	Allow Corp Apps	10.0.0.0/8	8.8.8.8	-1	25	UDP	DENY	18	0.23
12	Deny External Ac...	10.0.0.0/8	8.8.8.8	-1	445	UDP	LOG	74	
13	Log Intranet	172.16.0.0/16	10.0.1.0/24	-1	3389	UDP	DENY	97	9.17
14	Block Spam	192.168.1.0/24	192.168.100.0/24	-1	3389	UDP	LOG	89	0.23
15	Allow Backup	172.16.0.0/16	10.0.1.0/24	-1	0	UDP	ALLOW	11	0.23

Add Rule Remove Rule Save Rules Load Rules

The Tab Rulemanagement Showing the default Rules and rules loaded from File

Network Rule Engine

Rules Management | Packet Capture | Packet Analysis | System Logs

WAN Miniport (Network Monitor)

WAN Miniport (Network Monitor)

WAN Miniport (IPv6)

WAN Miniport (IP)

Intel(R) Centrino(R) Advanced-N 6235

Microsoft Wi-Fi Direct Virtual Adapter #2

Microsoft Wi-Fi Direct Virtual Adapter

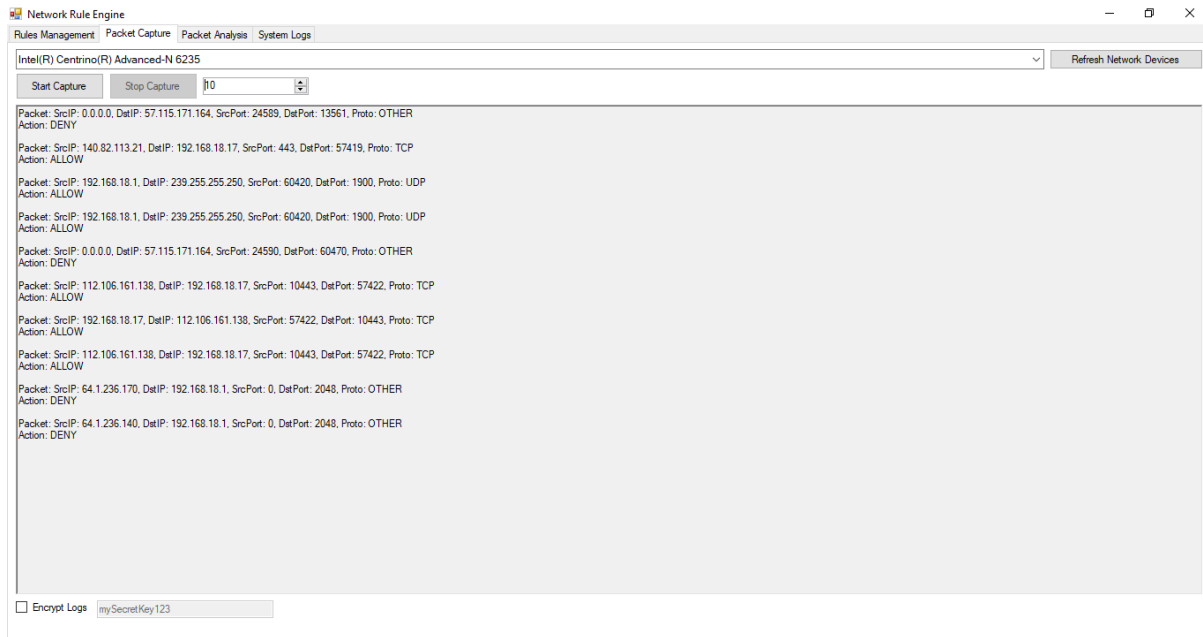
Adapter for loopback traffic capture

Realtek PCIe GbE Family Controller

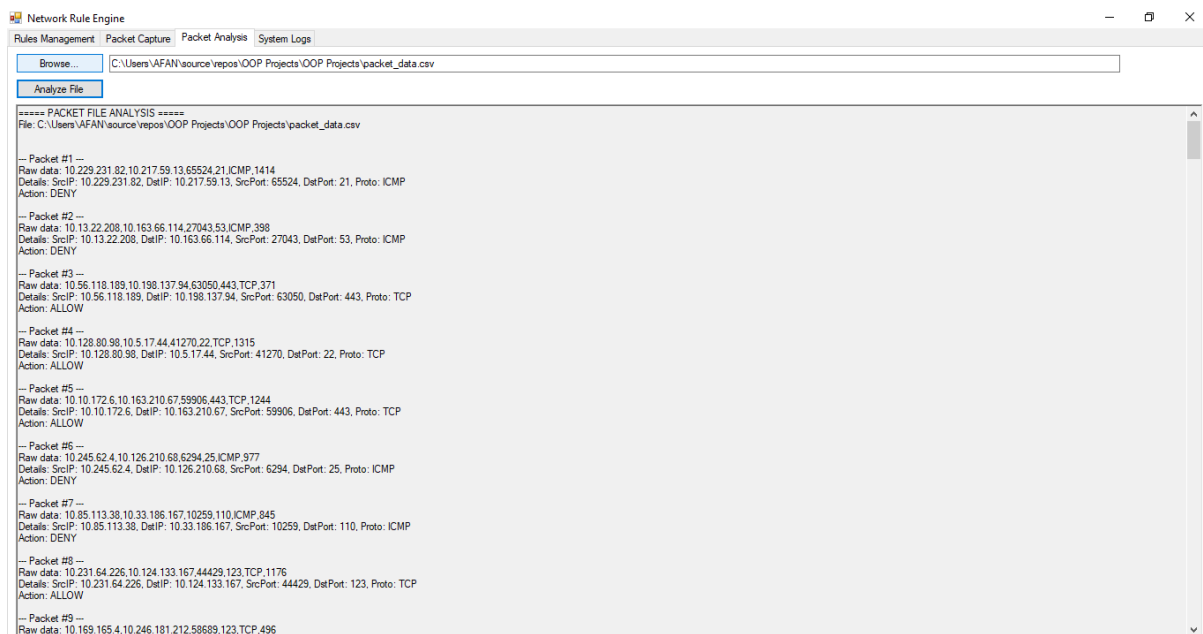
Refresh Network Devices

Encrypt Logs mySecretKey123

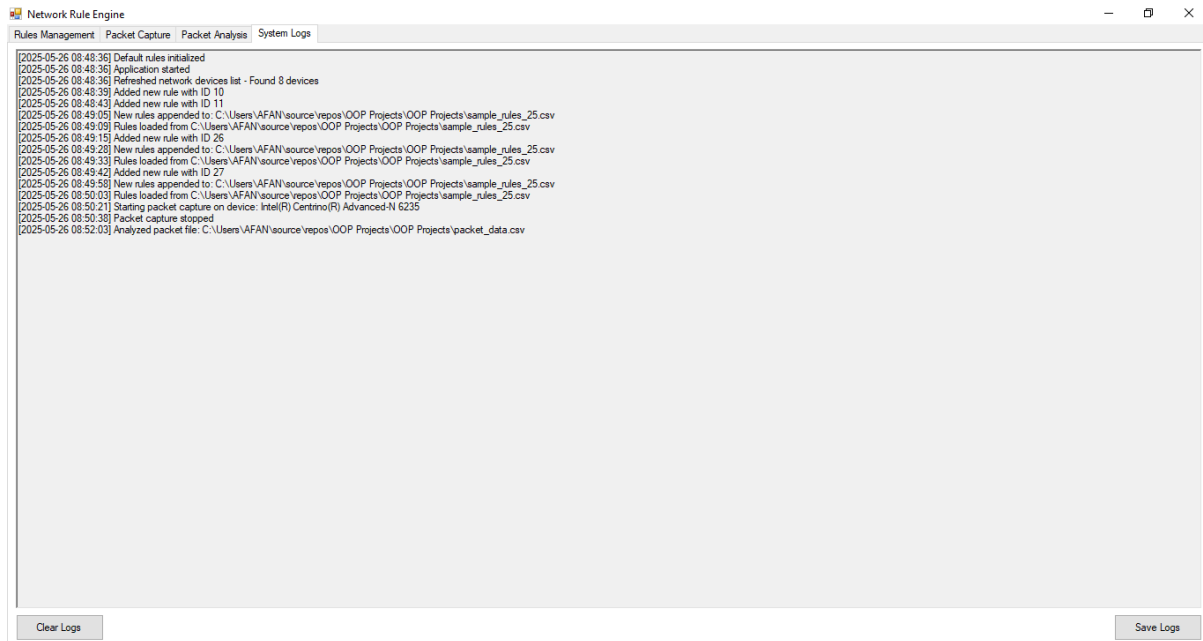
The Packet Capture Tab Showing the available network devices for Packet Capture.



The Packet Capture Tab showing packets that are captured and Showing action on them.



The Packet analysis Tab showing the packets that are analyzed on basis of rule. This Tab allows user to give their packets for analysis.



The systems log showing all the logs on the system.

References

External Libraries

WinPcap / Npcap

The project uses an external packet capture library (such as WinPcap or Npcap) to enable low-level network packet capture and device enumeration.

NET Framework / Windows Forms

- System.Windows.Forms For GUI components.
- System.Threading For background packet capture
- System::IO For string conversion between .NET and C++.
- Microsoft::VisualBasic For user prompts (e.g., encryption key input).

C++/CLI Interop

- **msclrinteropmarshal_as** Used for converting between .NET strings (System::String^) and standard C++ strings (std::string).